

CS3300 - Tools of the Trade #5

Student Demo Notes | Google Cloud Platform & App Engine

Big Picture

Last lecture, we built and ran a Spring Boot REST API on our local machine. In this activity, we move the same application to Google App Engine so it can be accessed through a public URL.

The main progression is:

```
LOCAL:  Browser -> localhost:8081 -> Spring Boot -> Controller -> Service -> JSON
CLOUD:  Browser -> Internet -> App Engine -> Spring Boot -> Controller -> Service -> JSON
```

1. GCP Project and Billing

- Open <https://console.cloud.google.com/>.
- Use the project selector to create or select your CS3300 GCP project.
- Verify that the project is associated with the Billing Account for Education and that your \$50 course credit has been applied.

Key idea: A GCP project is the cloud-side administrative container for resources such as App Engine, APIs, permissions, and billing. Your Spring Boot project and your GCP project are different things.

2. Create the App Engine Application

- In the Google Cloud Console, search for App Engine and open it.
- If this is the first App Engine use for the project, choose Create Application/Create App.
- Choose an appropriate supported region. This is a one-time choice for the project.
- Use the App Engine default service account. Do not create a new service account for this demo.
- Complete the App Engine setup.

Key idea: Creating the App Engine application creates the cloud-side environment. Your Spring Boot code is not deployed yet.

3. Enable Required APIs

- Open APIs & Services -> Library.
- Search for Cloud Build API and enable it if needed.
- Search for App Engine Admin API and enable it if needed.

Key idea: Google Cloud services are exposed through APIs. For today, these APIs support the App Engine deployment workflow.

4. Install and Verify Google Cloud CLI

Install the Google Cloud CLI only if you are deploying from your local computer. Official installation instructions:

<https://cloud.google.com/sdk/docs/install>

On Windows, download and run the installer. After installation, close and reopen PowerShell or the VS Code terminal.

Open the terminal on the same computer where your Spring Boot project is stored and run:

```
gcloud --version
```

Key idea: The gcloud command is the command-line interface for managing Google Cloud from your computer.

5. Connect the CLI to Your GCP Project

```
gcloud init
```

- Sign in when prompted.
- Choose the Google account that has access to the CS3300 GCP project.
- Select your CS3300 project.

Verify the selected project:

```
gcloud config get-value project
```

6. Open the Spring Boot Project

Open the same Spring Boot project you used in TOTT #4. Do not create a new application.

Key idea: The application code stays essentially the same. We are changing the execution environment from local to cloud.

7. Add App Engine Configuration

In the project root, next to pom.xml, create app.yaml:

```
runtime: java21
env: standard
service: default
```

Remember the role of each configuration file:

File	Purpose
pom.xml	Maven build, dependencies, and packaging
application.properties	Spring Boot application configuration
app.yaml	App Engine deployment/runtime configuration

8. Build and Test

```
mvn clean package
```

Key idea: Maven still builds and packages the Java application. App Engine is simply the deployment target.

9. Deploy to App Engine

Make sure your terminal is in the folder containing app.yaml, then run:

```
gcloud app deploy
```

Read the project/application/region information before accepting the deployment prompt.

10. Verify the Public Deployment

```
gcloud app browse
```

Open the deployed /students endpoint:

```
https://<app-engine-url>/students
```

Compare:

```
LOCAL: http://localhost:8081/students
```

```
CLOUD: https://<app-engine-url>/students
```

Key idea: Same controller. Same service. Same Java objects. Same JSON. Different execution environment.

11. Cloud Shell (Alternative Workflow)

Cloud Shell is a browser-based terminal provided by Google. It already includes gcloud, so you do not need to install the Google Cloud CLI there.

```
git clone <repository-url>
cd <repository-folder>
gcloud config get-value project
mvn clean package
gcloud app deploy
```

Key idea: For a team project, you can clone the GitHub repository into Cloud Shell and deploy from there instead of installing gcloud locally.

12. Project 1 Google APIs

Project 1 can use Google Maps Platform services:

API	Question it answers	Project 1 connection
Places	What places match?	Search and place details
Maps	Where are they?	Interactive map and markers
Routes	How do I travel between them?	Directions, distance, travel time

For API credentials, API keys are common for Maps Platform APIs. OAuth is used when a user authorizes access to protected user data. Service accounts provide identities for application-to-application access.

Never commit unrestricted API keys or other secrets to GitHub. Restrict credentials appropriately and keep secrets out of source code.

13. Cost Awareness and Cleanup

Cloud resources can incur costs. For a temporary demo project, use the Console to manage the project and shut it down when it is no longer needed. Do not shut down your Project 1 deployment before the course no longer needs it.